
UNIT-III

Functions and Program structure: User defined functions, inter function communication, Scope and Lifetime of variables, Storage classes-auto, register, static, extern, type qualifiers. The C pre-processor. Recursive functions.

Designing Structured Programs :

Structured programming is a programming technique in which a larger program is divided into smaller subprograms to make it easy to understand, easy to implement and makes the code reusable etc,. The structured programming enables code reusability. Code reusability is a method of writing code once and using it many times. Using structured programming technique, we write the code once and use it many times. Structured programming also makes the program easy to understand, improves the quality of the program, easy to implement and reduces time.

In C, the structured programming can be designed using functions concept. Using functions concept, we can divide larger program into smaller subprograms and these subprograms are implemented individually. Every subprogram or function in C is executed individually.

Introduction to Functions

When we write a program to solve a larger problem, we divide that larger problem into smaller subproblems and are solved individually to make the program easier. In C, this concept is implemented using functions.

Functions are used to divide a larger program into smaller subprograms such that program becomes easy to understand and easy to implement.

A function is defined as follows...

Definition

Function is a subpart of program used to perform specific task and is executed individually.

In short:

- A function is a group of statements that together perform a task.
- A program segment that carries out some specific, well-defined task
 - C functions are basic building blocks in a program.

(OR)

- A large C program is divided into basic building blocks called C function.
- C function contains set of instructions enclosed by “{ }”
- which performs specific operation in a C program.
- Actually, Collection of these functions creates a C program

Every C program must contain atleast one function called main(). However a program may also contain other functions.

Every function in C has the following...

- Function Declaration (Function Prototype)
- Function Definition
- Function Call

Function Declaration

The function declaration tells the compiler about function name, datatype of the return value and parameters. The function declaration is also called as function prototype. The function declaration is performed before main function or inside main function or inside any other function.

Function declaration syntax -

returnType functionName(parametersList);

- In the above syntax, returnType specifies the datatype of the value which is sent as a return value from the function definiton.
- The functionName is a user defined name used to identify the function uniquely in the program.
- The parametersList is the data values that are sent to the function definition.

Function Definition

- The function definition provides the actual code of that function.
- The function definition is also known as body of the function.
- The actual task of the function is implemented in the function definition. That means the actual instructions to be performed by a function are written in function definition. The actual instructions of a function are written inside the braces "{ }". The function definition is performed before main function or after main function.

Function definition syntax -

```
returnType functionName(parametersList)
{
    Actual code...
}
```

Function Call

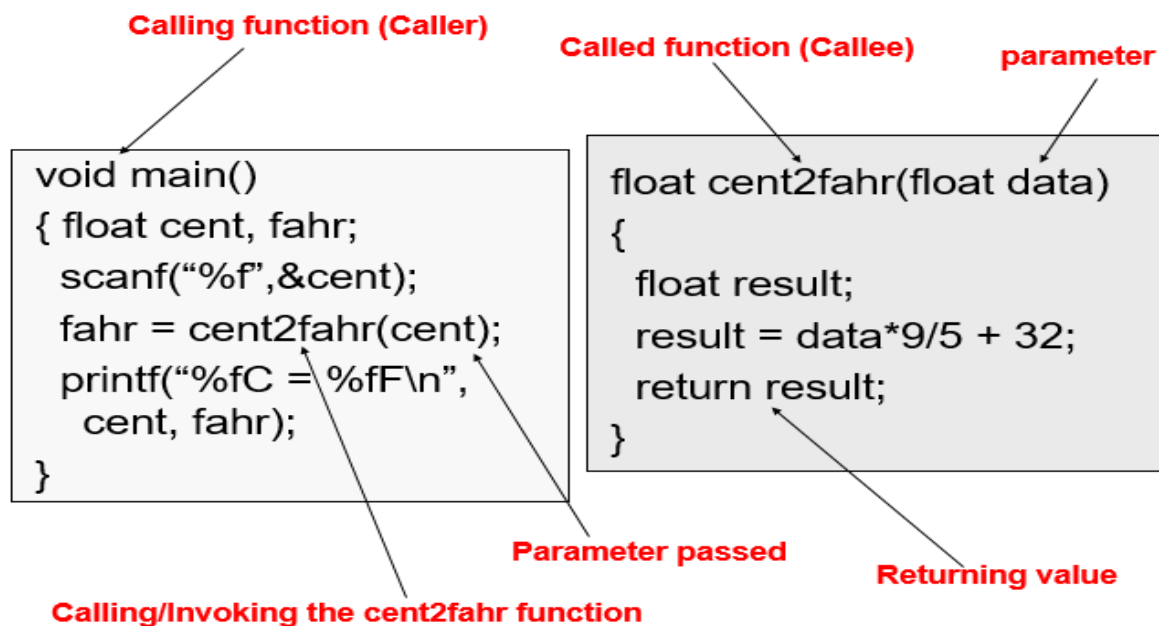
The function call tells the compiler when to execute the function definition. When a function call is executed, the execution control jumps to the function definition where the actual code gets executed and returns to the same functions call once the execution completes. The function call is performed inside main function or inside any other function or inside the function itself.

Function call syntax -

functionName(parameters);

functions aspects	syntax
function declaration	return_type function_name (argument list);
function call	function_name (arguments list);
function definition	Return_type function_name (arguments list) { Body of function; }

Example:



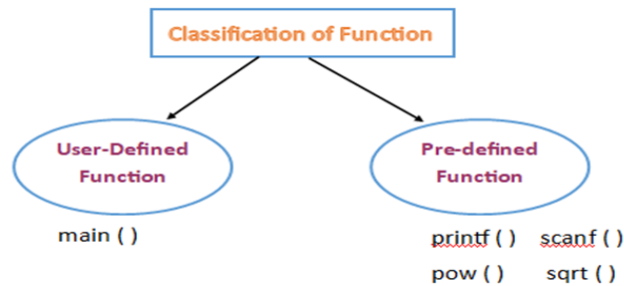
Advantages of Functions

- Using functions we can implement modular programming.
- Functions makes the program more readable and understandable.
- Using functions the program implementation becomes easy.
- Once a function is created it can be used many times (code re-usability).
- Using functions larger program can be divided into smaller modules.

Types of Functions

In C Programming Language, based on providing the function definition, functions are divided into two types. Those are as follows...

- **System Defined Functions**
- **User Defined Functions**



1. System Defined Functions

The C Programming Language provides pre-defined functions to make programming easy. These pre-defined functions are known as system defined functions.

The system defined function is defined as follows...

Definition:

The function whose definition is defined by the system is called as system defined function.

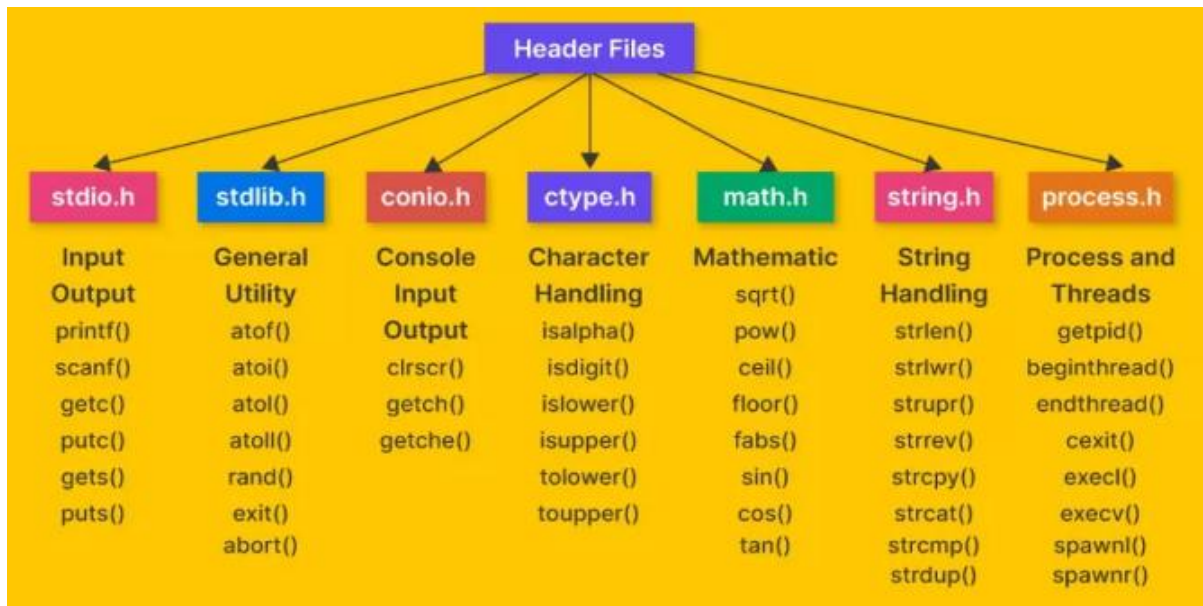
The system defined functions are also called as Library Functions or Standard Functions or Pre-Defined Functions.

The implementation of system defined functions is already defined by the system.

- Library functions in C language are inbuilt functions which are grouped together and placed in a common place called library. Each library function in C performs specific operation.
- used to handle tasks such as mathematical computations, I/O processing, string handling etc.
- These functions are defined in the header file. When you include the header file, these functions are available for use.

For example:

- The printf() is a standard library function to send formatted output to the screen (display output on the screen). This function is defined in "stdio.h" header file.
- There are other numerous library functions defined under "stdio.h", such as scanf(), printf(), getchar() etc. Once you include "stdio.h" in your program, all these functions are available for use.



2. User Defined Functions:

In C programming language, users can also create their own functions. The functions that are created by users are called as user defined functions.

The user defined function is defined as follows...

Definition:

The function whose definition is defined by the user is called as user defined function.

In C every user defined function must be declared and implemented. Whenever we make function call the function definition gets executed.

For example, consider the following program in which we create a function called addition with two parameters and a return value.

Program:

```
#include <stdio.h>

void main(){
    int num1, num2, result ;
    int addition(int,int) ; // function declaration
    printf("Enter any two integer numbers : ") ;
    scanf("%d%d", &num1, &num2);
    result = addition(num1, num2) ; // function call
    printf("SUM = %d", result);
}
```

```
int addition(int a, int b) // function definition
{
return a+b ;
}
```

In the above example program,

the function declaration statement "int addition(int,int)" tells the compiler that there is a function with name addition which takes two integer values as parameters and returns an integer value. The function call statement takes the execution control to the additon() definition along with values of num1 and num2. Then function definition executes the code written inside it and comes back to the function call along with return value.

In the concept of functions, the function call is known as **"Calling Function"** and the function definition is known as **"Called Function"**.

When we make a function call, the execution control jumps from calling function to called function. After executing the called function, the execution control comes back to calling function from called function. When the control jumps from calling function to called function it may carry one or more data values called "Parameters" and while coming back it may carry a single value called "return value". That means the data values transferred from calling function to called function are called as Parameters and the data value transferred from called function to calling function is called Return value.

Based on the data flow between the calling function and called function, the functions are classified as follows...

- Function without Parameters and without Return value
- Function with Parameters and without Return value
- Function without Parameters and with Return value
- Function with Parameters and with Return value

Function without Parameters and without Return value

In this type of functions there is no data transfer between calling function and called function. Simply the execution control jumps from calling function to called function and executes called function, and finally comes back to the calling function.

For example, consider the following program...

```
#include <stdio.h>

void main(){
void addition() ; // function declaration
addition() ; // function call
}

void addition() // function definition
```

```

{
int num1, num2 ;
printf("Enter any two integer numbers : ") ;
scanf("%d%d", &num1, &num2);
printf("Sum = %d", num1+num2 ) ;
}

```

Function with Parameters and without Return value

In this type of functions there is data transfer from calling function to called function (parameters) but there is no data transfer from called function to calling function (return value). The execution control jumps from calling function to called function along with the parameters and executes called function, and finally comes back to the calling function.

For example, consider the following program...

```

#include <stdio.h>

void main(){
int num1, num2 ;

void addition(int, int) ; // function declaration

printf("Enter any two integer numbers : ") ;
scanf("%d%d", &num1, &num2);
addition(num1, num2) ; // function call
}

void addition(int a, int b) // function definition
{
printf("Sum = %d", a+b ) ;
}

```

Function without Parameters and with Return value

In this type of functions there is no data transfer from calling function to called function (parameters) but there is data transfer from called function to calling function (return value).

The execution control jumps from calling function to called function and executes called function, and finally comes back to the calling function along with a return value.

For example, consider the following program...

```

#include <stdio.h>

void main(){

```

```

int result ;

int addition() ; // function declaration

result = addition() ; // function call

printf("Sum = %d", result) ;

}

int addition() // function definition

{

int num1, num2 ;

printf("Enter any two integer numbers : ") ;

scanf("%d%d", &num1, &num2);

return (num1+num2) ;

}

```

Function with Parameters and with Return value

In this type of functions there is data transfer from calling function to called function (parameters) and also from called function to calling function (return value). The execution control jumps from calling function to called function along with parameters and executes called function, and finally comes back to the calling function along with a return value.

For example, consider the following program...

```

#include <stdio.h>

void main(){

int num1, num2, result ;

int addition(int, int) ; // function declaration

printf("Enter any two integer numbers : ") ;

scanf("%d%d", &num1, &num2);

result = addition(num1, num2) ; // function call

printf("Sum = %d", result) ;

}

int addition(int a, int b) // function definition

{

return (a+b) ;

}

```

Function Working

- C program doesn't execute the statement in function until the function is called.
- When function is called the program can send the function information in the form of one or more argument.
- When the function is used it is referred to as the called function
- Functions often use data that is passed to them from the calling function
- Data is passed from the calling function to a called function by specifying the variables in a argument list.
- Argument list cannot be used to send data. Its only copy data/value/variable that pass from the calling function.
- The called function then performs its operation using the copies.

Example:

```
#include <stdio.h>

int fact(int);

void main() {
    int no,factorial;

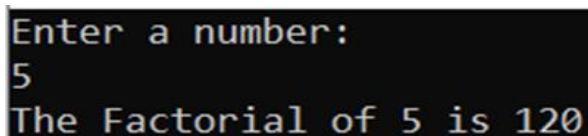
    printf("Enter a number: \n");

    scanf("%d",&no);

    factorial=fact(no);

    printf("The Factorial of %d is %d\n",no, factorial);
}

int fact(int n)
{
    int i, f=1;
    for(i=1;i<=n; i++)
    {
        f=f*i;
    }
    return f;
}
```

A screenshot of a terminal window showing the output of the C program. The text is as follows:

```
Enter a number:
5
The Factorial of 5 is 120
```

//Finding maximum number out of two numbers

```
#include <stdio.h>

int max(int num1, int num2);

int main ()
{
    int a = 200; int b = 800;
    int ret;
    ret = max(a, b);
    printf( "Max value is : %dn", ret );
    return 0; }

int max(int num1, int num2) {
    int result;
    if (num1 > num2)
        result = num1;
    else
        result = num2;
    return result; }
```

OUTPUT:

Max value is : 800

//Finding Average of three numbers

```
#include<stdio.h>

float average(float, float, float); //Function prototype

int main( )
{
    float a, b, c;
    printf("Enter three numbers please\n");
    scanf("%f, %f, %f",&a, &b, &c);
    printf("Avg of 3 numbers = %.3f\n", average(a, b, c) ); //Function call
    return 0;
}
```

```
float average(float x, float y, float z) //local variables x, y, z
{
float r; // local variable
r = (x+y+z)/3;
return r;
}
```

// Sample Greeting using functions

```
#include<stdio.h>

void hai ( ) //function definition
{
printf("welcome to functions \n");
printf("good morning");
}

int main( )
{
float a, b, c;
printf("main, welcome to functions \n");
hai ( ) //calling
printf("bye");
return 0;
}
```

Parameter Passing in C

When a function gets executed in the program, the execution control is transferred from calling function to called function and executes function definition, and finally comes back to the calling function. When the execution control is transferred from calling function to called function it may carry one or more number of data values. These data values are called as parameters.

Parameters are the data values that are passed from calling function to called function.

In C, there are two types of parameters and they are as follows...

- **Actual Parameters**
- **Formal Parameters**

The actual parameters are the parameters that are specified in calling function. The formal parameters are the parameters that are declared at called function. When a function gets executed, the copy of actual parameter values are copied into formal parameters.

In C Programming Language, there are two methods to pass parameters from calling function to called function and they are as follows...

- **Call by Value**
- **Call by Reference**

Call by Value

In call by value parameter passing method, the copy of actual parameter values are copied to formal parameters and these formal parameters are used in called function. The changes made on the formal parameters does not effect the values of actual parameters. That means, after the execution control comes back to the calling function, the actual parameter values remains same.

For example consider the following program...

```
#include <stdio.h>

void main()
{
    int num1, num2 ;
    void swap(int,int) ; // function declaration
    num1 = 10 ;
    num2 = 20 ;
    printf("\nBefore swap: num1 = %d, num2 = %d", num1, num2) ;
    swap(num1, num2) ; // calling function
    printf("\nAfter swap: num1 = %d\nnum2 = %d", num1, num2);
}

void swap(int a, int b) // called function
{
    int temp ;
    temp = a ;
    a = b ;
    b = temp ;
}
```

Output:

Before swap: num1 = 10, num2 = 20

After swap: num1 = 10, num2 = 20

In the above example program, the variables num1 and num2 are called actual parameters and the variables a and b are called formal parameters. The value of num1 is copied into a and the value of num2 is copied into b. The changes made on variables a and b does not effect the values of num1 and num2.

Call by Reference(or) pointers with Functions

In Call by Reference parameter passing method, the memory location address of the actual parameters is copied to formal parameters. This address is used to access the memory locations of the actual parameters in called function. In this method of parameter passing, the formal parameters must be pointer variables.

That means in call by reference parameter passing method, the address of the actual parameters is passed to the called function and is recieved by the formal parameters (pointers).

Whenever we use these formal parameters in called function, they directly access the memory locations of actual parameters. So the changes made on the formal parameters effects the values of actual parameters.

For example consider the following program...

```
#include <stdio.h>

void main(){
    int num1, num2 ;
    void swap(int *,int *) ; // function declaration
    num1 = 10 ;
    num2 = 20 ;
    printf("\nBefore swap: num1 = %d, num2 = %d", num1, num2) ;
    swap(&num1, &num2) ; // calling function
    printf("\nAfter swap: num1 = %d, num2 = %d", num1, num2);
}

void swap(int *a, int *b) // called function
{
    int temp ;
    temp = *a ;
    *a = *b ;
    *b = temp ;
}
```

Output:

Before swap: num1 = 10, num2 = 20

After swap: num1 = 20, num2 = 10

In the above example program, the addresses of variables num1 and num2 are copied to pointer variables a and b. The changes made on the pointer variables a and b in called function effects the values of actual parameters num1 and num2 in calling function.

Difference between call by value and reference

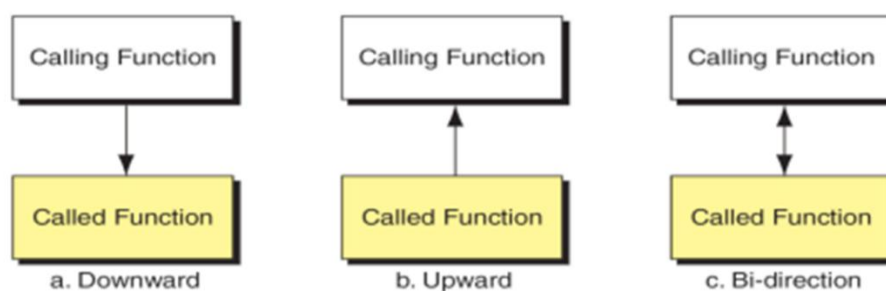
No.	Call by value	Call by reference
1	A copy of the value is passed into the function	An address of value is passed into the function
2	Changes made inside the function is limited to the function only. The values of the actual parameters do not change by changing the formal parameters.	Changes made inside the function validate outside of the function also. The values of the actual parameters do change by changing the formal parameters.
3	Actual and formal arguments are created at the different memory location	Actual and formal arguments are created at the same memory location

Inter Function Communication in C

When a function gets executed in the program, the execution control is transferred from calling function to called function and executes function definition, and finally comes back to the calling function. In this process, both calling and called functions have to communicate each other to exchange information. **The process of exchanging information between calling and called functions is called as inter function communication.**

In C, the inter function communication is classified as follows...

1. Downward Communication
2. Upward Communication
3. Bi-directional Communication



Downward Communication:

In this type of inter function communication, the data is transferred from calling function to called function but not from called function to calling function. The functions with parameters and without return value are considered under downward communication. In the case of downward communication, the execution control jumps from calling function to called function along with parameters and executes the function definition, and finally comes back to the calling function without any return value. For example consider the following program...

```
#include <stdio.h>

void main(){

int num1, num2 ;

void addition(int, int) ; // function declaration

num1 = 10 ;

num2 = 20 ;

printf("\nBefore swap: num1 = %d, num2 = %d", num1, num2) ;

addition(num1, num2) ; // calling function

}

void addition(int a, int b) // called function

{

printf("SUM = %d", a+b) ;

}
```

Output:

SUM = 30

Upward Communication:

In this type of inter function communication, the data is transferred from called function to calling function but not from calling function to called function. The functions without parameters and with return value are considered under upward communication. In the case of upward communication, the execution control jumps from calling function to called function without parameters and executes the function definition, and finally comes back to the calling function along with a return value.

For example consider the following program...

```
#include <stdio.h>

void main(){

int result ;

int addition() ; // function declaration
```

```

result = addition() ; // calling function
printf("SUM = %d", result) ;
}
int addition() // called function
{
int num1, num2 ;
num1 = 10;
num2 = 20;
return (num1+num2) ;
}

```

Output:

SUM = 30

Bi - Directional Communication:

In this type of inter function communication, **the data is transferred from calling function to called function and also from called function to calling function**. The functions with parameters and with return value are considered under bi-directional communication. In the case of bi-directional communication, the execution control jumps from calling function to called function along with parameters and executes the function definition, and finally comes back to the calling function along with a return value.

For example consider the following program...

```

#include <stdio.h>

void main(){
int num1, num2, result ;

int addition(int, int) ; // function declaration

num1 = 10 ;
num2 = 20 ;

result = addition(num1, num2) ; // calling function
printf("SUM = %d", result) ;
}

int addition(int a, int b) // called function
{
return (a+b) ;
}

```

```
}
```

Output:

SUM = 30

Scope of Variable in C

When we declare a variable in a program, it cannot be accessed against the scope rules. Variables can be accessed based on their scope. Scope of a variable decides the portion of a program in which the variable can be accessed. Scope of the variable is defined as follows...

Definition:

Scope of a variable is the portion of the program where a defined variable can be accessed.

The variable scope defines the visibility of variable in the program. Scope of a variable depends on the position of variable declaration.

In C programming language, a variable can be declared in three different positions and they are as follows...

1. Before the function definition (Global Declaration)
2. Inside the function or block (Local Declaration)
3. In the function definition parameters (Formal Parameters)

Before the function definition (Global Declaration):

Declaring a variable before the function definition (outside the function definition) is called global declaration. The variable declared using global declaration is called global variable. The global variable can be accessed by all the functions that are defined after the global declaration. That means the global variable can be accessed anywhere in the program after its declaration. The global variable scope is said to be file scope.

```
#include <stdio.h>
```

```
int num1, num2 ;
```

```
void main(){
```

```
void addition() ;
```

```
void subtraction() ;
```

```
void multiplication() ;
```

```
num1 = 10 ;
```

```
num2 = 20 ;
```

```
printf("num1 = %d, num2 = %d", num1, num2) ;
```

```
addition() ;
```

```
subtraction() ;
```

```

multiplication() ;
}

void addition()
{
int result ;
result = num1 + num2 ;
printf("\naddition = %d", result) ;
}

void subtraction()
{
int result ;
result = num1 - num2 ;
printf("\nsubtraction = %d", result) ;
}

void multiplication()
{
int result ;
result = num1 * num2 ;
printf("\nmultiplication = %d", result) ;
}

```

Output:

num1 = 10, num2 = 20

addition = 30

subtraction = -10

multiplication = 200

In the above example program, the variables num1 and num2 are declared as global variables. They are declared before the main() function. So, they can be accessed by function main() and other functions that are defined after main(). In the above example, the functions main(), addition(), subtraction() and multiplication() can access the variables num1 and num2.

Inside the function or block (Local Declaration):

Declaring a variable inside the function or block is called local declaration. The variable declared using local declaration is called local variable. The local variable can be accessed only

by the function or block in which it is declared. That means the local variable can be accessed only inside the function or block in which it is declared.

```
#include <stdio.h>

void main(){
void addition() ;

int num1, num2 ;

num1 = 10 ;
num2 = 20 ;

printf("num1 = %d, num2 = %d", num1, num2) ;

addition() ;

getch() ;
}

void addition()
{
int sumResult ;

sumResult = num1 + num2 ;

printf("\naddition = %d", sumResult) ;

}
```

Output:

ERROR

The above example program shows an error because, the variables num1 and num2 are declared inside the function main(). So, they can be used only inside main() function and not in addition() function.

In the function definition parameters (Formal Parameters):

The variables declared in function definition as parameters have local variable scope. These variables behave like local variables in the function. They can be accessed inside the function but not outside the function.

```
#include <stdio.h>

void main(){
void addition(int, int) ;

int num1, num2 ;

num1 = 10 ;
```

```
num2 = 20 ;  
addition(num1, num2) ;  
}  
void addition(int a, int b)  
{  
    int sumResult ;  
    sumResult = a + b ;  
    printf("\naddition = %d", sumResult) ;  
}
```

Output:

addition = 30

Lifetime

The lifetime of a variable is how long it exists in memory and holds its value. There are two types of variable lifetimes: static and automatic. Static variables remain active until the program ends, while automatic variables activate when they are called.

Storage Classes

the storage class in the C language for determining the visibility, lifetime, initial value, and memory location of any given variable. The storage classes define the visibility (scope) and the lifetime of any function/ variable within a C program. These classes precede the type that they are going to modify.

Types of Storage Classes:

There are four different types of storage classes that we use in the C language:

- Automatic Storage Class
- External Storage Class
- Static Storage Class
- Register Storage Class

Use of Storage Class:

A variable given in a C program will have two of the properties: storage class and type. Here, type refers to any given variable's data type, while the storage class determines that very variable's lifetime, visibility, and also its scope.

Class	Name of Class	Place of Storage	Scope	Default Value	Lifetime
auto	Automatic	RAM	Local	Garbage Value	Within a function
extern	External	RAM	Global	Zero	Till the main program ends. One can declare it anywhere in a program.
static	Static	RAM	Local	Zero	Till the main program ends. It retains the available value between various function calls.
register	Register	Register	Local	Garbage Value	Within the function

Automatic Storage Class:

It is also known as the auto storage class, and it acts as the default storage class for all the variables that are local in nature.

For example,

```
{
int mount;
auto int month;
}
```

Look at the example that we used above- it defines two of the variables in the very same storage class. One can use 'auto' only within the functions- or the local variables.

Features of automatic variables:

- The allocation of memory for these variables occurs automatically during the runtime.
- The scope of an automatic variable is limited to that block in which we are defining them.
- The visibility of these variables is also limited to that block in which we are defining them.
- The initialization of these variables is, by default, a garbage value.
- The memory that is assigned to an automatic variable gets free when it exits from a block.
- We use the keyword auto to define the automatic variables.
- Any local variable that exists in the C language is, by default, automatic in nature.

Example:

```
#include <stdio.h>

int main()
```

```

{
int p; //auto
char q;
float r;
printf(“%d %c %f”,p,q,r); // to print the initial default value of the automatic variables p, q, and
r.
return 0;
}

```

The output generated here would be:

garbage garbage garbage

Let us look at another example,

```

#include <stdio.h>
int main()
{
int p = 10,i;
printf(“%d “,++p);
{
int p = 20;
for (i=0;i<3;i++)
{
printf(“%d “,p); // 20 will be printed here 3 times because it is the given local value of p
}
}
printf(“%d “,p); // 11 will be printed here since the scope of p = 20 has finally ended.
}

```

The output generated here would be:

11 20 20 20 11

External Storage Class:

The extern storage class is used to give a reference of a global variable that is visible to ALL the program files. When you use 'extern', the variable cannot be initialized however, it points the variable name at a storage location that has been previously defined.

When you have multiple files and you define a global variable or function, which will also be used in other files, then `extern` will be used in another file to provide the reference of defined variable or function. Just for understanding, `extern` is used to declare a global variable or function in another file.

The `extern` modifier is most commonly used when there are two or more files sharing the same global variables or functions as explained below.

main.c (File 1) –

```
#include <stdio.h>

int count ;

variable void write_variable();

main() {
count = 38;
write_variable();
}
```

support.c (File 2) –

```
#include <stdio.h>

variable int count;

void write_variable(void) {
printf("The variable for the program is %d\n", count);
}
```

In this example, we have used the `extern` keyword for declaring the variable available in the second file. On the other hand, its definition actually resides in the `main.c` (that is its first file).

result as follows:

The variable for the program is 38

static Storage Class:

The static storage class instructs the compiler to keep a local variable in existence during the life-time of the program instead of creating and destroying it each time it comes into and goes out of scope. Therefore, making local variables static allows them to maintain their values between function calls.

The static modifier may also be applied to global variables. When this is done, it causes that variable's scope to be restricted to the file in which it is declared.

In C programming, when static is used on a global variable, it causes only one copy of that member to be shared by all the objects of its class.

Example

```
#include<stdio.h>

void sum()
{
static int x = 20;
static int y = 34;
printf(“%d %d \n”,x,y);
x++;
y++;
}

void main()
{
int a;
for(a = 0; a< 3; a++)
{
sum(); // Given static variables will hold their values between the various function calls.
}
}
```

The output generated here would be:

20 34

21 35

22 36

register Storage Class:

The register storage class is used to define local variables that should be stored in a register instead of RAM. This means that the variable has a maximum size equal to the register size (usually one word) and can't have the unary '&' operator applied to it (as it does not have a memory location).

The register should only be used for variables that require quick access such as counters. It should also be noted that defining 'register' does not mean that the variable will be stored in a register. It means that it MIGHT be stored in a register depending on hardware and implementation restrictions.

example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
register int x; // A variable x has memory allocation in the CPU register. Here, the initial value of x, by default, is 0.
```

```
printf("%d",x);
```

```
}
```

Thus, the output generated here would be:

0

Type Qualifiers in C

Type qualifiers are keywords that can be used to change a data type's behavior in the C programming language. These qualifiers can be used to describe a variable or pointer's constancy, volatility, restrictions, and others.

These type qualifiers are as follows:

1. Const: If any variable is declared as a const in the code and is initialized with a value, its value cannot be further modified in any part of the code. It results in a compile-time error if the programmer attempts to modify the value of such variables, which are declared const.

example

```
#include <stdio.h>
```

```
int main() {
```

```
    const int y=10;
```

```
    printf("The value of const variable y is %d",y);
```

```
    y=11;
```

```
    printf("The value of variable after modification is %d",y);
```

```
    return 0;
```

```
}
```

Output:

The code results in a compile time error since the user is trying to modify the value of const variable 'y' after its initialization.

2. Volatile: The keyword "volatile" can be used to describe a variable whose value is subject to sudden change. It is typically used for variables that can be accessed by hardware, updated by several threads or processes, or both. The volatile qualifier informs the compiler that any

reads or writes to the variable should not be optimized away since the variable's value might change without the program's permission.

Syntax:

volatile <data_type><variable_name>

3. Restrict: Use the restrict qualifier to specify that a pointer is an exclusive method of accessing a specific object or area of memory. Assuming that no other pointer may access the same memory region, enables the compiler to generate code that is more effective. In performance-sensitive code, such as numerical algorithms, the restrict qualifier is often used.

An example

```
Void fun(int *restrict m, int *restrict n);
```

4. Atomic: When a variable may be read or written atomically, meaning without interference from other threads or processes, it is indicated by this type of qualifier. It is frequently used in multithreaded programming to provide synchronized and reliable data access.

Syntax to declare a variable:

_Atomic <data_type><identifier>

Recursive Functions in C

In C programming language, function calling can be made from main() function, other functions or from same function itself. The recursive function is defined as follows...

Definition:

A function called by itself is called recursive function.

- Recursion is a process in which a function calls itself directly or indirectly.
- Recursion is the process which comes into existence when a function calls a copy of itself to work on a smaller problem.
- Any function which calls itself is called recursive function, and such function calls are called recursive calls.
- Recursion involves several numbers of recursive calls. However, it is important to impose a termination condition of recursion.
- recursion may be applied to sorting, searching, and traversal problems.

The recursive functions should be used very carefully because, when a function called by itself it enters into infinite loop. And when a function enters into the infinite loop, the function execution never gets completed. We should define the condition to exit from the function call so that the recursive function gets terminated.

When a function is called by itself, the first call remains under execution till the last call gets invoked. Every time when a function call is invoked, the function returns the execution control to the previous function call.

Example-1:

```

#include <stdio.h>

int fun(int n) {
    if(n==1)
        return 1;
    else
        return 1+fun(n-1);
}

int main( ) {
    int n=3;
    printf("%d", fun(n));
    return 0;
}

```

Example-2

```

#include <stdio.h>

int fun(int n) {
    if(n==0)
        return 1;
    else
        return 7+fun(n-2);
}

int main( ) {
    printf("%d", fun(4));
    return 0;
}

```

Recursive Function-Pseudocode format

```

if (test_for_base)
{
    return some_value;
}

else if (test_for_another_base)

```

```

{
    return some_another_value;
}
else
{
    // Statements;
    recursive call;
}

```

Example-3

```

#include <stdio.h>
#include<conio.h>
int factorial( int ) ;
void main( )
{
    int fact, n ;
    printf("Enter any positive integer: ");
    scanf("%d", &n) ;
    fact = factorial( n ) ;
    printf("Factorial of %d is %d", n, fact) ;
}
int factorial( int n )
{
    int temp ;
    if( n == 0)
        return 1 ;
    else
        temp = n * factorial( n-1 ) ; // recursive function call
    return temp ;
}

```

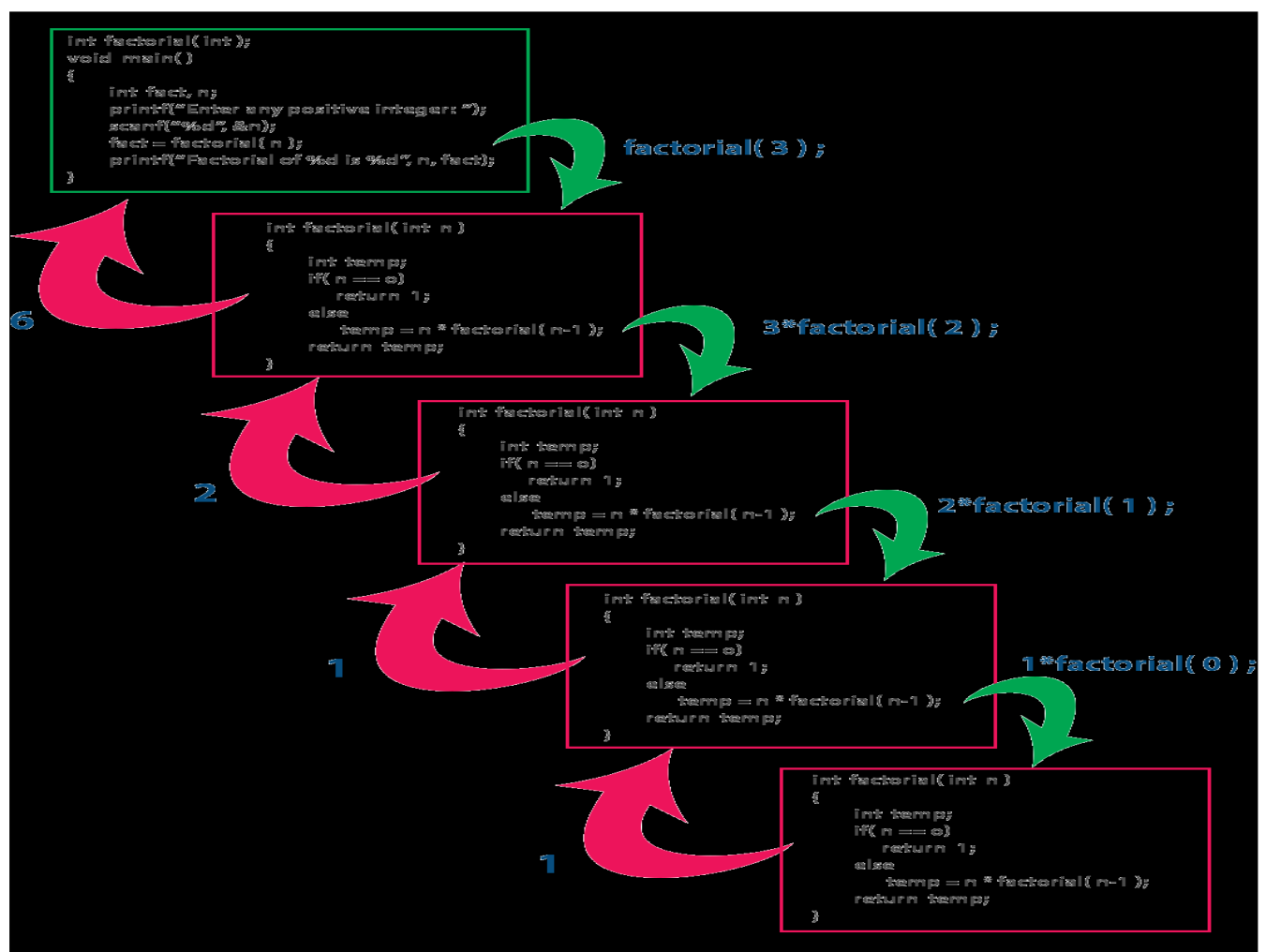
Output:

Enter any positive integer: 3

Factorial of 3 is 6

In the above example program, the factorial() function call is initiated from main() function with the value 3. Inside the factorial() function, the function calls factorial(2), factorial(1) and factorial(0) are called recursively. In this program execution process, the function call factorial(3) remains under execution till the execution of function calls factorial(2), factorial(1) and factorial(0) gets completed. Similarly the function call factorial(2) remains under execution till the execution of function calls factorial(1) and factorial(0) gets completed. In the same way the function call factorial(1) remains under execution till the execution of function call factorial(0) gets completed.

The complete execution process of the above program is shown in the following figure...



Another Method:

```
#include<stdio.h>
```

```
int fact(int);
```

```
int main()
```

```
{
```

```

int n,f;
printf("\n enter n value");
scanf("%d",&n);
f=fact(n);
Printf("factorial of %d is %d",n,f);
return 0;
}
int fact(int n)
{
if (n==0) {
    return 0;
}
else if ( n == 1) {
    return 1;
}
else {
    return n*fact(n-1);
}
}

```

We can understand the program of the recursive method call by the figure given below:

```

return 5 * factorial(4) = 120
└─ return 4 * factorial(3) = 24
    └─ return 3 * factorial(2) = 6
        └─ return 2 * factorial(1) = 2
            └─ return 1 * factorial(0) = 1

```

javaTpoint.com

$1 * 2 * 3 * 4 * 5 = 120$

Fig: Recursion

Method-1: Fibonacci Series in C Using Recursion

```

#include <stdio.h>

int fibonacci(int n) {
    if(n == 0)
        return 0;
    else if(n == 1)
        return 1;
    else
        return (fibonacci(n-1) +
                fibonacci(n-2));
}

int main() {
    int n;
    printf("Enter the number of terms\n");
    scanf("%d", &n);
    printf("Fibonacci Series: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }
}

```

```
    return 0;
}
```

Method-2: Fibonacci Series in C Using Recursion

```
#include <stdio.h>
```

```
void printFibonacci(int n) {
    static int n1 = 0, n2 = 1, n3;
```

```
    if(n > 0) {
        n3 = n1 + n2;
        n1 = n2;
        n2 = n3;
        printf("%d ", n3);
        printFibonacci(n-1);
    }
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of elements: ");
```

```
    scanf("%d", &n);
```

```
    printf("Fibonacci Series: ");
```

```
    printf("%d %d ", 0, 1); // printing 0 and 1
```

```
    printFibonacci(n-2); // n-2 because 2 numbers are already printed
```

```
    return 0;
```

```
}
```

GCD and LCM of two numbers

```
#include <stdio.h>
```

```

int gcd(int a, int b) {
    if (b == 0) // Base case
        return a;
    return gcd(b, a % b);
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int num1, num2;
    printf("Enter two integers: ");
    scanf("%d%d", &num1, &num2);

    printf("GCD of %d and %d is %d\n", num1, num2, gcd(num1, num2));

    printf("LCM of %d and %d is %d\n", num1, num2, lcm(num1, num2));
    return 0;
}

```

limitations of recursion

Recursion in C has several limitations, including:

Slow

Recursive programs are slower than non-recursive programs because they require the program to save and retrieve its current state each time a function call is made.

Memory usage

Recursive programs require more memory to store intermediate states in the stack.

Stack overflows

Recursive functions can cause stack overflows when the call stack is full and the recursive function has not returned a value.

Confusing

Recursion can be confusing and difficult to understand unless you are familiar with how it works.

C Preprocessor:

The C Preprocessor is a tool that runs before your actual C program is compiled.

It processes lines that start with `#` and performs tasks like:

- Including header files
- Macro substitution
- Conditional compilation
- File guarding

Types of Preprocessor Directives:

Directive	Meaning
-----	-----
`#include`	Includes files (like stdio.h)
`#define`	Creates macros or constants
`#ifdef`, `#ifndef`, `#endif`	Used for conditional compilation
`#undef`	Removes a defined macro
`#pragma`	Compiler-specific instructions

1. `#include` Example

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello World!");  
    return 0;  
}
```

`#include <stdio.h>` tells the preprocessor to insert the contents of the stdio.h file.

2. `define` (Macro) Example

```
#include <stdio.h>
```

```
#define PI 3.14
```

```
#define AREA(r) (PI * r * r)
```

```
int main() {  
    int r = 5;  
    printf("Area: %.2f", AREA(r));  
    return 0;  
}
```

Here:

- `PI` is replaced by `3.14`
- `AREA(r)` is replaced by `(3.14 \* r \* r)` before compiling.

3. Conditional Compilation Example

```
#include <stdio.h>

#define DEBUG

int main() {
#ifdef DEBUG
    printf("Debug mode ON\n");
#else
    printf("Debug mode OFF\n");
#endif
    return 0;
}
```

If `DEBUG` is defined → prints `Debug mode ON`

If undefined → prints `Debug mode OFF`

4. Header Guard Example

Used to prevent multiple inclusion of the same header file:

```
#ifndef MY_HEADER
#define MY_HEADER

void printMessage() {
    printf("Header included!\n");
}

#endif
```

Working:

When you compile a program:

1. Preprocessor runs first → expands macros, includes files.
2. Sends modified code to the compiler.
3. Compiler generates machine code.

Summary

Feature	Directive
-----	-----
File Inclusion	<code>`#include`</code>
Define constants/macros	<code>`#define`</code>
Conditional compile	<code>`#ifdef`</code> , <code>`#ifndef`</code> , <code>`#endif`</code> , <code>`#else`</code>
Remove define	<code>`#undef`</code>
Compiler instruction	<code>`#pragma`</code>